A19 Maxilerator

Parameterizable Output-Stationary INT8 Systolic Array AI Accelerator

Discord Name	Affiliation	Role	Experience
irene_raphael	Technical University of Munich	Team Lead (Feeder)	Master's Student
amxlanil	Technical University of Munich	Team Member (Controller)	Master's Student
muhammedrabin.	Technical University of Munich	Team Member (Output Processor)	Master's Student
myx_on_57134	National Taiwan University of Science and Technology	Team Member (Memory)	Master's Student
maroonchocopie	Indian Institute of Technology, Delhi	Team Member (MAC, Systolic Array)	Master's Student

Video Link: https://drive.google.com/file/d/1-m7rP2tFc2djv29NrM6ZzOCDwGAntJ3e/view?usp=sharing

Project Information, Design Objectives & Constraints

Project Context

- SSCS Chipathon 2026 — **Track A: Foundational Building Blocks**
- Target PDK: **GF180MCU**
- Target slot: **Slot A**, approx. **1.25 mm²**

What We Are Building

This project implements a compact **8×8 output-stationary systolic array accelerator** for tiled INT8 matrix multiplication, targeting TinyML / CNN-style edge inference workloads.
The design is structured as a reusable digital IP block rather than only a one-time chip demo.

"The goal is not only to build an accelerator, but to package the systolic-array compute engine as a reusable, parameterizable, and verification-ready digital IP block."

What Makes This Design Stand Out

- **Three-level reusability:** reusable arithmetic primitives, reusable accelerator_core, and wrapper-level integration paths
- **Throughput-oriented ping-pong buffering:** overlaps tile loading with computation to improve effective throughput
- **Parameterized architecture:** array size, accumulator width, and interface width can be adapted for different workloads, precision needs, area budgets, and integration targets
- **Output-stationary dataflow:** keeps partial sums local inside MAC units, reducing intermediate data movement
- **Verification-friendly RTL:** clean datapath/control separation and module-level testability

Main Design Objective

Build a compact, reusable, and verifiable **INT8 matrix-multiplication accelerator** suitable for open-source silicon and TinyML-style edge inference workloads.

Functional Objectives

- Implement an **8×8 output-stationary systolic array**
- Support signed **INT8 × INT8 multiplication**
- Use **21-bit accumulation** for safe partial-sum storage
- Support tiled matrix multiplication for matrices larger than 8×8
- Use two **128×8 SRAM macros** in ping-pong mode
- Target **25 MHz** operation on GF180MCU
- Fit within **Slot A** area constraints

RTL / Reusability Objectives

- Keep mac_unit and systolic_array reusable as arithmetic building blocks
- Provide accelerator_core as the main reusable IP boundary
- Separate the Chipathon physical interface from the reusable accelerator logic
- Support wrapper-level integration, including an AXI4-Stream reference path
- Keep datapath, memory, controller, and output logic clearly separated

Design Constraints

- Slot A area budget: approx. 1.25 mm²
- 22 physical-facing top-level pins
- Target clock: 25 MHz
- SRAM macros are single-port and 8-bit wide
- Must support tiled computation beyond native 8×8 array size

System Overview

RTL System Overview

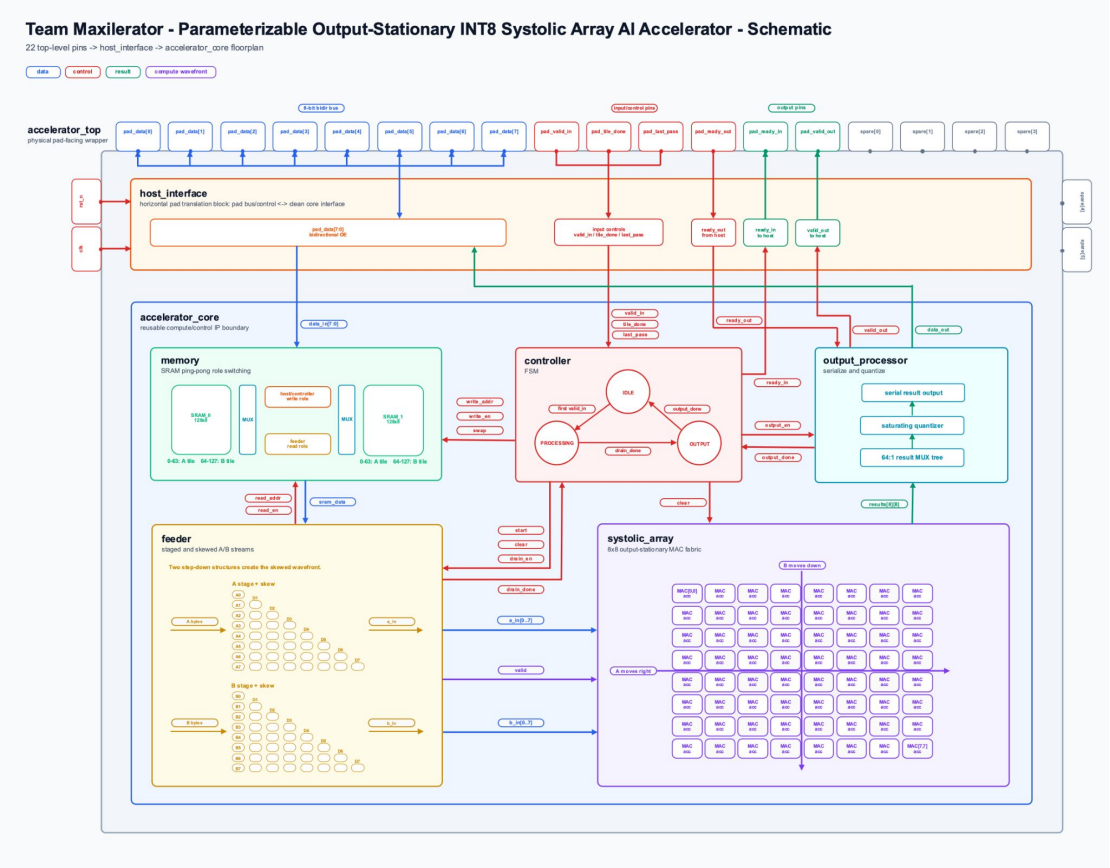
- accelerator_top: chip-facing wrapper with **22 pins**
- host_interface: translates pad protocol to core signals
- accelerator_core: reusable compute/control IP boundary
- memory: two **128×8 SRAMs** for ping-pong buffering
- feeder: generates skewed A/B streams
- systolic_array: **8×8 grid of 64 reusable** mac_units
- output_processor: serializes saturated INT8 results

Tile-Level Processing Flow

1. Host sends 128-byte A/B tile
2. Feeder creates skewed A/B streams from SRAM
3. Systolic array performs local MAC accumulation
4. Next tile loads into inactive SRAM in parallel
5. Controller swaps SRAM roles after tile_done
6. On last_pass, feeder drains remaining wavefront
7. Output processor serializes 64 INT8 results

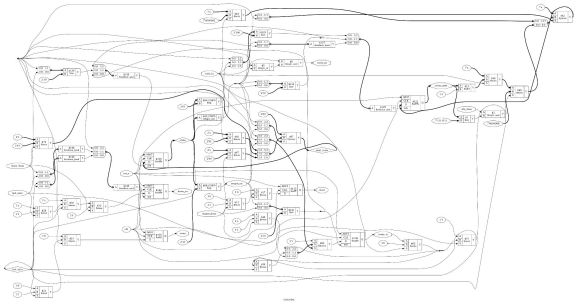
Key RTL Idea

- Data path: memory → feeder → systolic_array → output_processor
- Control path: controller FSM → write_en / swap / clear / output_en
- Ping-pong buffering overlaps tile load and compute
- Output-stationary flow keeps partial sums inside MACs

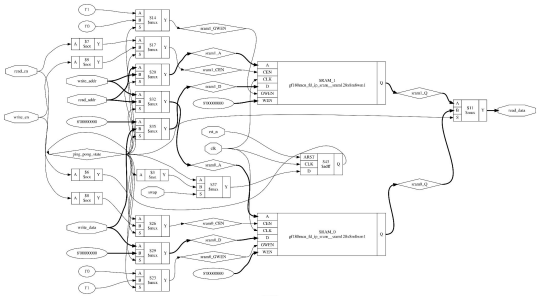


RTL Schematics

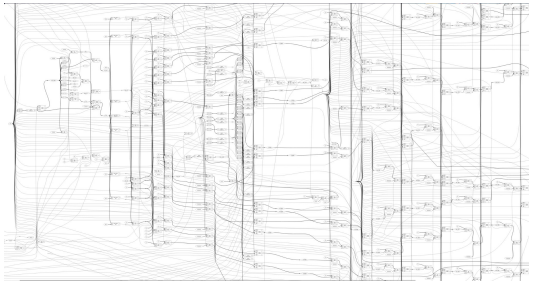
controller.sv



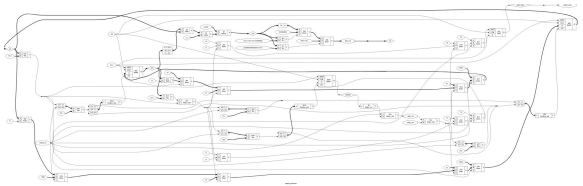
memory.sv



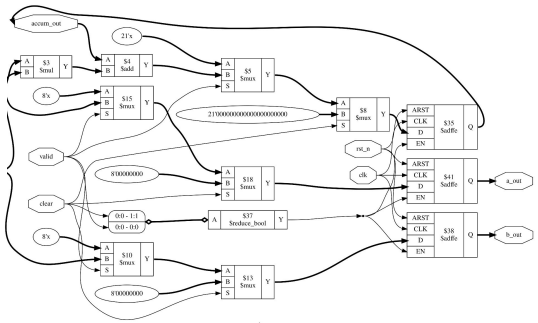
feeder.sv



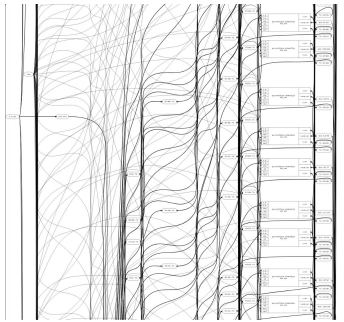
output_processor.sv



mac_unit.sv



systolic_array.sv



RTL Design Decisions & Feasibility Analysis

Key Design Decisions

Design Choice	Reason
Output-stationary	Keeps partial sums local inside MAC units
INT8 precision	Suitable for compact TinyML-style inference
8x8 array size	Balances compute parallelism and Slot A area
21-bit accumulation	Provides safe range for INT8 partial sums
Ping-pong SRAM	Overlaps tile loading with computation
128-byte tile format	Matches 64 A bytes + 64 B bytes per tile
Serialized output	Reduces physical pin count
Separate host interface	Keeps accelerator_core reusable

Handshake Protocol Summary

Signal	Role
valid_in	Host provides a valid input byte
ready_in	Tile-level permission to start a new tile transfer
tile_done	Completion of 128-byte tile transfer
last_pass	Marks the final computation pass
swap	Switches active/inactive SRAM roles
clear	Clears feeder state and MAC accumulators
output_en	Enables output serialization
valid_out	Output byte is valid
ready_out	Host is ready to accept output byte
output_done	All 64 output bytes transferred

Area Feasibility

Component	Estimated Area / Slot Impact
64 MAC units	~808,636 µm² / 64.8%
SRAM_0	~116,119 µm² / 9.3%
SRAM_1	~116,119 µm² / 9.3%
Feeder	~54,062 µm² / 4.3%
Output processor	~46,071 µm² / 3.7%
Controller + host interface	<1% combined
Total at 70% utilization	~1,143,718 µm² / 91.6% Slot A

Accumulator Width Trade-Off

Width	Max K	Coverage	Extra Area	Slot A	Time @25MHz
21	32	ECG / biosignals	0 µm²	91.6%	0.464 ms
22	64	Gesture / keyword	15.6k µm²	92.8%	3.177 ms
23	128	Tiny image cls.	31.1k µm²	94.1%	23.276 ms
24	256	Small speech	46.7k µm²	95.3%	177.644 ms
25	512	Medium CNN	62.2k µm²	96.6%	1.387 s
26	1024	Large CNN / ResNet	77.8k µm²	97.8%	10.958 s
27	2048	BERT-small	93.3k µm²	99.1%	1.45 min
28	4096	GPT-2 small	108.9k µm²	100.3%	11.58 min

Project Status, Verification Strategy & Open Questions

Current Project Status

Area	Status
Architecture	Defined and documented
System block diagram	Completed
Module hierarchy	Defined
RTL interfaces	Defined at signal level
Core RTL modules	Implemented and under refinement
Basic verification	Completed for main functional paths
Corner-case testing	Ongoing
Module synthesis	Initial synthesis checks completed
Area estimation	Preliminary estimate prepared
Physical design	Next phase after RTL freeze

Ongoing / Planned Verification Coverage

Module / Area	Verification Focus
mac_unit	Signed INT8 MAC, accumulator clear/reset
systolic_array	8x8 result correctness and accumulation flow
memory	SRAM read/write and ping-pong role switching
feeder	Skew timing, restart behavior, drain phase
controller	FSM transitions, swap, clear, output control
output_processor	Saturation, serialization, output backpressure
accelerator_core	End-to-end tiled matrix multiplication
protocol edges	ready_in, tile_done, last_pass, ready_out
fault cases	reset during operation, invalid/stalled input

Verification Status

- Basic module-level and functional-path tests completed
- cocotb-based verification flow is being expanded
- Python/NumPy reference model used for expected matrix results
- Current focus is **rigorous corner-case testing**
- Verification is being extended to cover fault conditions and protocol edge cases

End-to-End Test Scenarios

- 8×8 single-tile matrix multiplication
- Multi-tile operation for larger matrices
- Back-to-back tile transfers
- tile_done and last_pass timing checks
- ready_in tile-permission behavior
- Output backpressure using ready_out
- Reset/clear during active operation
- Stalled or delayed input transfer

Questions for Reviewers

- Is **70% utilization** a realistic target for this design in GF180MCU, considering routing congestion, SRAM placement, and the 8×8 MAC array?
- Is the area estimate reasonable for proceeding toward RTL freeze?
- Is the RTL module partitioning suitable for Track A?
- Is the architecture mature enough to move toward synthesis and physical design?